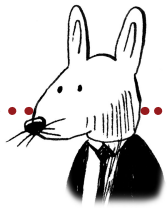
how to picture neural networks: in your head and on paper

7



In this chapter

- Correlation summarization
- Simplified visualization
- Seeing the network predict
- Visualizing using letters instead of pictures
- Linking variables
- The importance of visualization tools

> Numbers have an important story to tell. They rely on you to give them a clear and convincing voice.
>
> —Stephen Few, IT innovator, teacher, and consultant

It's time to simplify

It's impractical to think about everything all the time. Mental tools can help.

Chapter 6 finished with a code example that was quite impressive. Just the neural network contained 35 lines of incredibly dense code. Reading through it, it's clear there's a lot going on; and that code includes over 100 pages of concepts that, when combined, can predict whether it's safe to cross the street.

I hope you're continuing to rebuild these examples from memory in each chapter. As the examples get larger, this exercise becomes less about remembering specific letters of code and more about remembering concepts and then rebuilding the code based on those concepts.

In this chapter, this construction of efficient concepts in your mind is exactly what I want to talk about. Even though it's not an architecture or experiment, it's perhaps the most important value I can give you. In this case, I want to show how I summarize all the little lessons in an efficient way in my mind so that I can do things like build new architectures, debug experiments, and use an architecture on new problems and new datasets.

Let's start by reviewing the concepts you've learned so far.

This book began with small lessons and then built layers of abstraction on top of them. We began by talking about the ideas behind machine learning in general. Then we progressed to how individual linear nodes (or *neurons*) learned, followed by horizontal groups of neurons (layers) and then vertical groups (stacks of layers). Along the way, we discussed how learning is actually just reducing error to 0, and we used calculus to discover how to change each weight in the network to help move the error in the direction of 0.

Next, we discussed how neural networks search for (and sometimes create) correlation between the input and output datasets. This last idea allowed us to overlook the previous lessons on how individual neurons behaved because it concisely summarizes the previous lessons. The sum total of the neurons, gradients, stacks of layers, and so on lead to a single idea: neural networks find and create correlation.

Holding onto this idea of correlation instead of the previous smaller ideas is important to learning deep learning. Otherwise, it would be easy to become overwhelmed with the complexity of neural networks. Let's create a name for this idea: the *correlation summarization*.

Correlation summarization

This is the key to sanely moving forward to more advanced neural networks.

Correlation summarization

Neural networks seek to find direct and indirect correlation between an input layer and an output layer, which are determined by the input and output datasets, respectively.

At the 10,000-foot level, this is what all neural networks do. Given that a neural network is really just a series of matrices connected by layers, let's zoom in slightly and consider what any particular weight matrix is doing.

Local correlation summarization

Any given set of weights optimizes to learn how to correlate its input layer with what the output layer says it should be.

When you have only two layers (input and output), the weight matrix knows what the output layer says it should be based on the output dataset. It looks for correlation between the input and output datasets because they're captured in the input and output layers. But this becomes more nuanced when you have multiple layers, remember?

Global correlation summarization

What an earlier layer says it should be can be determined by taking what a later layer says it should be and multiplying it by the weights in between them. This way, later layers can tell earlier layers what kind of signal they need, to ultimately find correlation with the output. This cross-communication is called *backpropagation*.

When global correlation teaches each layer what it should be, local correlation can optimize weights locally. When a neuron in the final layer says, "I need to be a little higher," it then proceeds to tell all the neurons in the layer immediately preceding it, "Hey, previous layer, send me higher signal." They then tell the neurons preceding them, "Hey. Send us higher signal." It's like a giant game of telephone—at the end of the game, every layer knows which of its neurons need to be higher and lower, and the local correlation summarization takes over, updating the weights accordingly.

The previously overcomplicated visualization

While simplifying the mental picture, let's simplify the visualization as well.

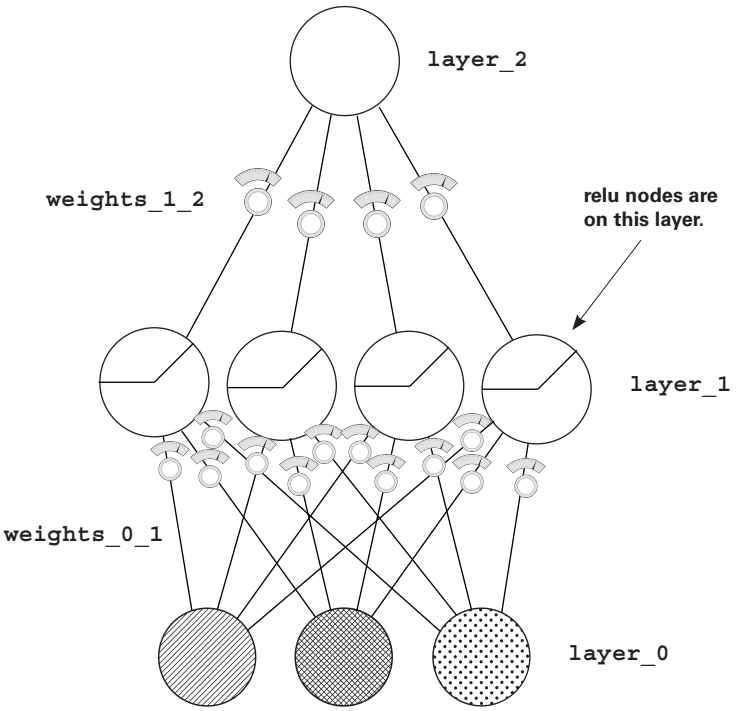
At this point, I expect the visualization of neural networks in your head is something like the picture shown here (because that's the one we used). The input dataset is in `layer_0`, connected by a weight matrix (a bunch of lines) to `layer_1`, and so on. This was a useful tool to learn the basics of how collections of weights and layers come together to learn a function.

But moving forward, this picture has too much detail. Given the correlation summarization, you already know you no longer need to worry about how individual weights are updated. Later layers already know how to communicate to earlier layers and tell them, "Hey, I need higher signal" or "Hey, I need lower signal." Truth be told, you don't really care about the weight values anymore, only that they're behaving as they should, properly capturing correlation in a way that generalizes.

To reflect this change, let's update the visualization on paper. We'll also do a few other things that will make sense later. As you know, the neural network is a series of weight matrices. When you're using the network, you also end up creating vectors corresponding to each layer.

In the figure, the weight matrices are the lines going from node to node, and the vectors are the strips of nodes. For example, `weights_1_2` is a matrix, `weights_0_1` is a matrix, and `layer_1` is a vector.

In later chapters, we'll arrange vectors and matrices in increasingly creative ways, so instead of all this detail showing each node connected by each weight (which gets hard to read if we have, say, 500 nodes in `layer_1`), let's instead think in general terms. Let's think of them as vectors and matrices of arbitrary size.

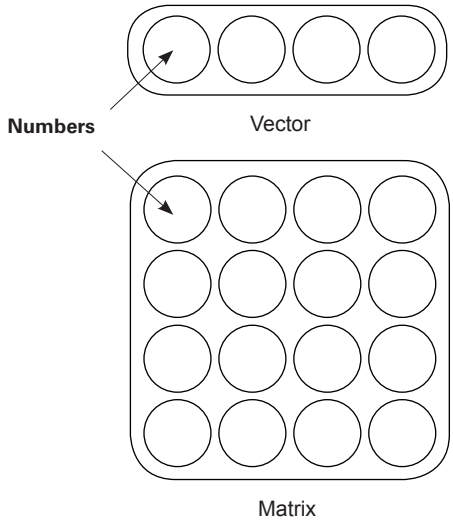


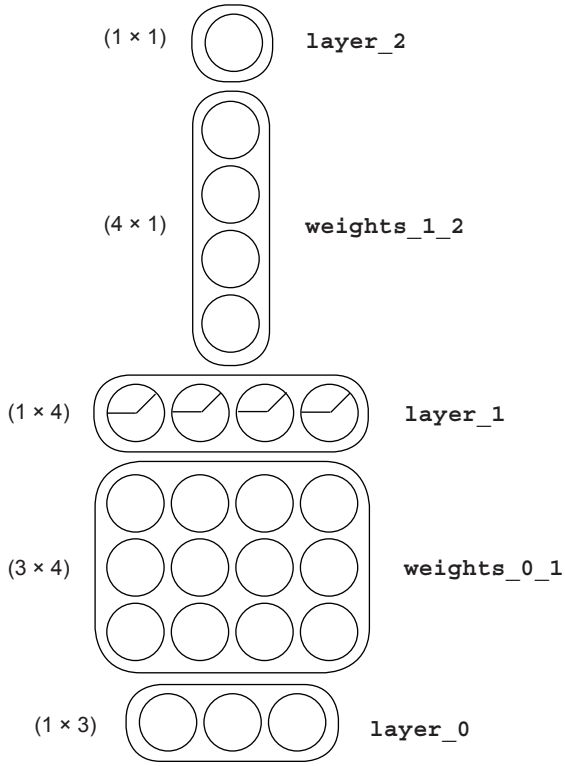
The simplified visualization

Neural networks are like LEGO bricks, and each brick is a vector or matrix.

Moving forward, we'll build new neural network architectures in the same way people build new structures with LEGO pieces. The great thing about the correlation summarization is that all the bits and pieces that lead to it (backpropagation, gradient descent, alpha, dropout, mini-batching, and so on) don't depend on a particular configuration of the LEGOs. No matter how you piece together the series of matrices, gluing them together with layers, the neural network will try to learn the pattern in the data by modifying the weights between wherever you put the input layer and the output layer.

To reflect this, we'll build all the neural networks with the pieces shown at right. The strip is a vector, the box is a matrix, and the circles are individual weights. Note that the box can be viewed as a "vector of vectors," horizontally or vertically.





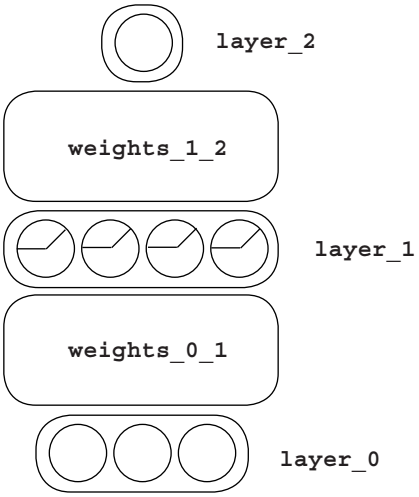
The big takeaway

The picture at left still gives you all the information you need to build a neural network. You know the shapes and sizes of all the layers and matrices. The detail from before isn't necessary when you know the correlation summarization and everything that went into it. But we aren't finished: we can simplify even further.

Simplifying even further

The dimensionality of the matrices is determined by the layers.

In the previous section, you may have noticed a pattern. Each matrix's dimensionality (number of rows and columns) has a direct relationship to the dimensionality of the layers before and after them. Thus, we can simplify the visualization even further.



Consider the visualization shown at right. We still have all the information needed to build a neural network. We can infer that `weights_0_1` is a (3 × 4) matrix because the previous layer (`layer_0`) has three dimensions and the next layer (`layer_1`) has four dimensions. Thus, in order for the matrix to be big enough to have a single weight connecting each node in `layer_0` to each node in `layer_1`, it must be a (3 × 4) matrix.

This allows us to start thinking about the neural networks using the correlation summarization. All this neural network will to do is adjust the weights to find correlation between `layer_0` and `layer_2`. It will do this using all the methods mentioned so far in this book. But the different configurations of weights and layers between the input and output layers have a strong impact on whether the network is successful in finding correlation (and/or how fast it finds correlation).

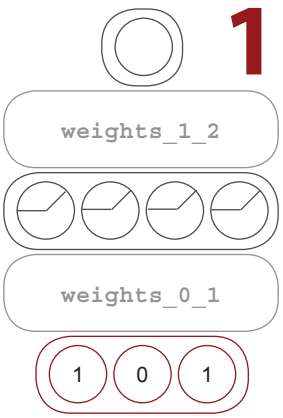
The particular configuration of layers and weights in a neural network is called its *architecture*, and we'll spend the majority of the rest of this book discussing the pros and cons of various architectures. As the correlation summarization reminds us, the neural network adjusts weights to find correlation between the input and output layers, sometimes even inventing correlation in the hidden layers. Different architectures *channel signal to make correlation easier to discover*.

> Good neural architectures channel signal so that correlation is easy to discover. Great architectures also filter noise to help prevent overfitting.

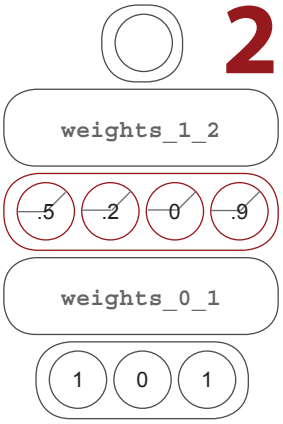
Much of the research into neural networks is about finding new architectures that can find correlation faster and generalize better to unseen data. We'll spend the vast majority of the rest of this book discussing new architectures.

Let's see this network predict

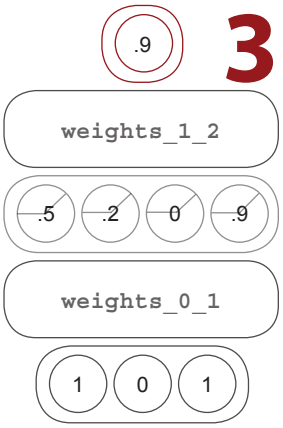
Let's picture data from the streetlight example flowing through the system.



In figure 1, a single datapoint from the streetlight dataset is selected. `layer_0` is set to the correct values.



In figure 2, four different weighted sums of `layer_0` are performed. The four weighted sums are performed by `weights_0_1`. As a reminder, this process is called *vector-matrix multiplication*. These four values are deposited into the four positions of `layer_1` and passed through the `relu` function (setting negative values to 0). To be clear, the third value from the left in `layer_1` would have been negative, but the `relu` function sets it to 0.



As shown in figure 3, final step performs a weighted average of `layer_1`, again using the vector-matrix multiplication process. This yields the number 0.9, which is the network's final prediction.

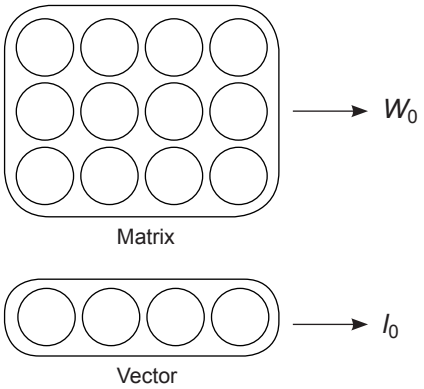
Review: Vector-matrix multiplication

Vector-matrix multiplication performs multiple *weighted sums* of a vector. The matrix must have the same number of rows as the vector has values, so that each column in the matrix performs a unique weighted sum. Thus, if the matrix has four columns, four weighted sums will be generated. The weightings of each sum are performed depending on the values of the matrix.

Visualizing using letters instead of pictures

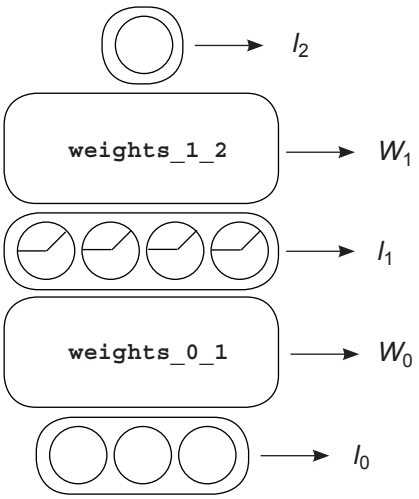
All these pictures and detailed explanations are actually a simple piece of algebra.

Just as we defined simpler pictures for the matrix and vector, we can perform the same visualization in the form of letters.



How do you visualize a *matrix* using math? Pick a capital letter. I try to pick one that's easy to remember, such as *W* for "weights." The little 0 means it's probably one of several *W*s. In this case, the network has two. Perhaps surprisingly, I could have picked any capital letter. The little 0 is an extra that lets me call all my weight matrices *W* so I can tell them apart. It's your visualization; make it easy to remember.

How do you visualize a *vector* using math? Pick a lowercase letter. Why did I choose the letter *l*? Well, because I have a bunch of vectors that are layers, I thought *l* would be easy to remember. Why did I choose to call it *l*-zero? Because I have multiple layers, it seems nice to make all them *l*s and number them instead of having to think of new letters for every layer. There's no wrong answer here.



If that's how to visualize matrices and vectors in math, what do all the pieces in the network look like? At right, you can see a nice selection of variables pointing to their respective sections of the neural network. But defining them doesn't show how they relate. Let's combine the variables via vector-matrix multiplication.

Linking the variables

The letters can be combined to indicate functions and operations.

Vector-matrix multiplication is simple. To visualize that two letters are being multiplied by each other, put them next to each other. For example:

Algebra	Translation
$l_0 W_0$	"Take the layer 0 vector and perform vector-matrix multiplication with the weight matrix 0."
$l_1 W_1$	"Take the layer 1 vector and perform vector-matrix multiplication with the weight matrix 1."

You can even throw in arbitrary functions like `relu` using notation that looks almost exactly like the Python code. This is crazy-intuitive stuff.

$l_1 = relu(l_0 W_0)$	"To create the layer 1 vector, take the layer 0 vector and perform vector-matrix multiplication with the weight matrix 0; then perform the `relu` function on the output (setting all negative numbers to 0)."
$l_2 = l_1 W_1$	"To create the layer 2 vector, take the layer 1 vector and perform vector-matrix multiplication with the weight matrix 1."

If you notice, the layer 2 algebra contains layer 1 as an input variable. This means you can represent the *entire neural network* in one expression by chaining them together.

$l_2 = relu(l_0 W_0) W_1$	Thus, all the logic in the forward propagation step can be contained in this one formula. Note: baked into this formula is the assumption that the vectors and matrices have the right dimensions.

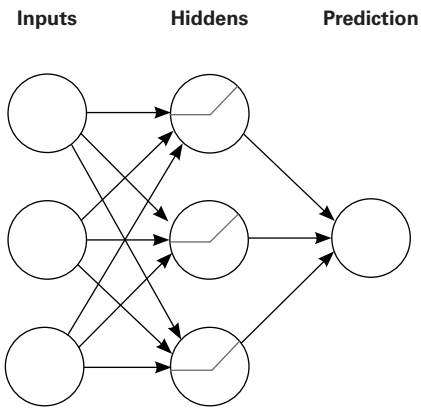
Everything side by side

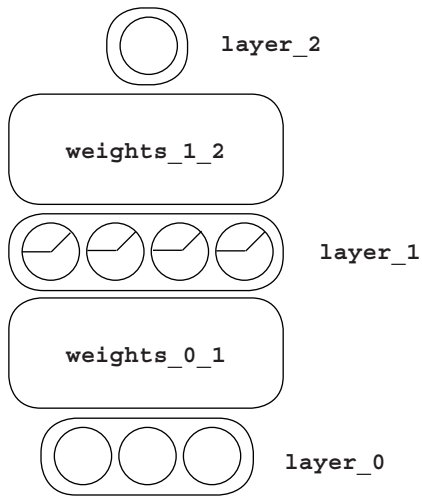
Let's see the visualization, algebra formula, and Python code in one place.

I don't think much dialogue is necessary on this page. Take a minute and look at each piece of forward propagation through these four different ways of seeing it. It's my hope that you'll truly grok forward propagation and understand the architecture by seeing it from different perspectives, all in one place.

```
layer_2 = relu(layer_0.dot(weights_0_1)).dot(weights_1_2)
```

$$l_2 = relu(l_0 W_0) W_1$$





The importance of visualization tools

We're going to be studying new architectures.

In the following chapters, we'll be taking these vectors and matrices and combining them in some creative ways. My ability to describe each architecture for you is entirely dependent on our having a mutually agreed-on language for describing them. Thus, please don't move beyond this chapter until you can clearly see how forward propagation manipulates these vectors and matrices, and how these various forms of describing them are articulated.

Key takeaway

Good neural architectures channel signal so that correlation is easy to discover. Great architectures also filter noise to help prevent overfitting.

As mentioned previously, a neural architecture controls how signal flows through a network. How you create these architectures will affect the ways in which the network can detect correlation. You'll find that you want to create architectures that maximize the network's ability to focus on the areas where meaningful correlation exists, and minimize the network's ability to focus on the areas that contain noise.

But different datasets and domains have different characteristics. For example, image data has different kinds of signal and noise than text data. Even though neural networks can be used in many situations, different architectures will be better suited to different problems because of their ability to locate certain types of correlations. So, for the next few chapters, we'll explore how to modify neural networks to specifically find the correlation you're looking for. See you there!